AI Collections Officer – Solution Design (Revised)

Nov 3, 2026 · @Alazar

An AI collections officer that a small lender can hire. It works on three levels: it **prevents** loans from going bad by acting before a payment is missed, **recovers** late loans by negotiating within the lender's rules and verifying every payment, and **steers** the portfolio by forecasting cash and testing policies before they are used. Every action is checked by code against the lender's rules, and its results are measured against plain reminders before anyone has to trust it.

In one example: a farmer's region has had a dry month and his last payment was partial. Ten days before his due date, the agent offers to move part of the payment to after harvest. He pays half through a link, the rest is scheduled, and a loan heading towards default stays current, with no officer involved.

Role, problem and value

Role: Collections officer. **Industry:** small lenders in emerging markets: microfinance institutions, savings and credit cooperatives, instalment sellers and digital lenders. **First market:** Ethiopia.

Losses start before collections does. Most lenders, and most collections software, start working once a borrower is already late. By then the cheapest moment to fix the problem has passed. One microfinance analysis notes that stepping in at 15 days past due rather than 75 can decide whether a loan is restructured or written off, and that manual tracking catches problems too late. Repayment also depends on things outside the borrower's control: Ethiopia's microfinance association names droughts, floods and erratic rainfall as threats to borrowers' ability to repay.

What small lenders struggle with:

- **Manual follow-up.** Loans live in spreadsheets, borrowers hold no account the lender can debit, and every reminder, payment check and plan is done by hand.
- **Restructuring that hides risk.** Constant restructuring can make a portfolio look healthy while the real problem grows, a risk lenders in Ethiopia have been warned about.
- **Cash pressure.** Ethiopia's microfinance sector, 59 institutions, lends more than it holds in deposits and depends on borrowing from banks and funders, so managers need to know what money is coming in.

- **Staff turnover.** Field officer turnover is high in microfinance, and knowledge about borrowers leaves with each officer.
- **Harsh collection practices.** Public shaming and contacting borrowers' friends and family are documented problems in East African digital lending. Fair treatment is part of what a lender buys.

Why a new product, not another chaser. Well-funded AI collections companies already automate borrower conversations for large lenders. This agent is different on three points: it is built for lenders without a lending platform, it acts before borrowers fall behind using outside signals as well as payment behaviour, and it gives the manager a forecast and a policy sandbox, not just a call list.

Value: fewer loans going bad, more recovered from those that do, less manual work, and decisions about policy and cash made with numbers.

I chose this role because I know the borrower's side of it: as a mobile engineer I built the loan and repayment screens of a digital lending app used by two Ethiopian banks. This agent covers the lender's side of the same journey.

Job description

The agent is designed like a hire: a clear set of duties, prioritised by their effect on money recovered, and targets it reports against every week.

Duties, in priority order:

Priority	Duty	Done today	What the agent does
1	Spot trouble early	Noticed once the borrower is already late	Watches payment behaviour and regional signals for every borrower, every day
1	Act before the due date	Rarely done; no time to contact everyone	Checks in early and offers a schedule that fits the borrower's income, within the playbook
2	Collect and verify payment	Officer checks statements and screenshots by hand	Sends payment links, matches payments automatically, checks receipts for fraud

Priority	Duty	Done today	What the agent does
2	Negotiate when a borrower cannot pay	Improvised and inconsistent	Agrees plans inside the lender's limits and checks each plan is realistic
3	Decide who needs a person	Officer's memory and gut feel	Hands over the cases that need a call or visit, with a brief, ranked by expected recovery
3	Advise the manager	End-of-month spreadsheet	Forecasts cash, tests policy changes in simulation, and reports weekly

Kept for people: calls and visits the agent recommends, disputes, hardship cases, legal steps, and any decision outside the playbook.

Targets the agent reports on:

- Collection rate: share of amount due that is paid on time.
- Loans saved: at-risk loans that were back on schedule after early action.
- Promise-kept rate: share of agreed plans that were paid as agreed.
- Handover rate: share of cases that needed a person.
- Messages per birr recovered: how much contact each result takes, so the agent does not win by messaging more.

How it works

Setup, once per lender. The manager uploads the loan spreadsheet as it is. Gemini maps each column to the system's fields (due date, amount, phone, region) and shows the mapping to confirm. The manager sets the playbook: negotiation limits, quiet hours, contact limits and how much the agent may do without approval. Borrowers receive a one-time invite to the lender's Telegram bot.

Every morning, started by Cloud Scheduler:

1. **Sync** new loans and payments, and match payments received overnight.
2. **Read the signals:** recent rainfall for each region with farming loans, and each borrower's payment behaviour.
3. **Score every loan** for risk, with plain reasons.
4. **Choose an action** for each borrower: wait, remind, check in early, offer a schedule change, negotiate, or hand over to the officer.

5. **Check** every action against the playbook in code.
6. **Act:** routine messages go out; sensitive ones wait for approval.

Between runs, the agent reacts to every reply, payment and receipt as it arrives.

Three borrowers, three paths:

- **Prevent. Abebe, a farmer due in 10 days.** Low rainfall in his area and a partial payment last month raise his risk before he is late. The agent checks in early and offers to move part of the payment to after harvest. He pays half through a Chapa link; the rest is scheduled with a follow-up.
- **Recover. Sara, a shop owner 5 days late.** She replies that she can pay only 1,000. That is below the playbook minimum, so the agent makes one counter-offer at the limit. She accepts and pays through the link, or sends a transfer receipt that the agent reads, checks and matches.
- **Hand over. Dawit, who broke his last plan and has gone silent.** The realism check shows another plan would only delay the problem. The agent stops negotiating and puts him on the officer's call list with his history and a suggested approach.

For the manager: a cash forecast, a policy sandbox to test changes before using them, a weekly performance report from the agent, and requests in plain language such as "Farmers in Sidama are struggling; what should we do?"

Agent design

Gemini decides, code enforces. The model reasons about each case and chooses actions through function calls. A policy layer written in code checks every action before a tool runs it. The agent can therefore be trusted in conversations about money: it cannot offer terms the lender has not allowed, whatever a borrower says.

Tools:

Tool	What it does	Checked by code
get_case	Loan, schedule, payments, plans, signals and conversation summary	Read only
send_message	Sends a message on Telegram or email	Quiet hours, daily contact cap, opt-out, tone check
create_payment_link	Creates a payment link for an exact amount	Cannot exceed the balance

Tool	What it does	Checked by code
propose_plan	Offers a schedule change or payment plan	Must fit the playbook and pass the realism check
verify_receipt	Reads a receipt image and checks it against payments	Duplicate and mismatch rules
record_promise	Saves a promise and books a Calendar follow-up	Valid amount and date
run_simulation	Runs a policy on a copy of the loan book	Read only; never changes live loans
draft_policy_change	Prepares a playbook change for the manager	Applied only after a manager approves
hand_over	Sends a case to the officer with a brief	Always allowed

Two kinds of work. Most work is per borrower: one case, a few tool calls, one action. Manager requests are multi-step: for "Farmers in Sidama are struggling; what should we do?", the agent finds the affected loans, builds two or three policy options, runs each through the simulator, compares recovery and cash impact, and drafts the best one for approval.

Memory. Each borrower has a case file: a running summary, open promises and plans, preferred language, channel and time, and what has worked with them. It belongs to the lender, not the officer, so nothing is lost when staff change.

Autonomy levels. The lender chooses: approve every message, approve only sensitive actions, or full autonomy within the playbook. Disputes, hardship and legal matters always go to a person.

Decision trace. Every run records what the agent saw, which tools it called, what the policy layer allowed or blocked, and why. Any message or decision can be opened to see its reasoning.

Markets as settings. Language, payment provider and channel are configuration. The demo uses English and Amharic, Chapa and Telegram; the same interfaces support other markets.

Prevent: early warning and early outreach

The agent's first job is to keep good loans from turning bad, because the cheapest default to fix is the one that has not happened yet.

Signals the agent watches:

- **Payment behaviour:** payments arriving later each month, partial payments, broken promises, a plan already in place.
- **Engagement:** slower replies, unread messages, a change of phone number.
- **Regional signals for farming loans:** rainfall over recent weeks compared with the usual level for that place and season, from the free Open-Meteo weather API, for the region recorded on each loan. Weather is used only for loans tied to farming, not for shops or salaried borrowers.

Risk score. Each loan gets a likelihood-to-pay score with its top reasons, for example "two late payments, partial payment last month, rainfall well below normal." On day one it is a transparent set of weighted rules, because a new lender has no history to learn from. As outcomes come in, the weights are refitted to that lender's own results, and the dashboard shows which signals actually predicted missed payments.

Early outreach. When risk rises before a due date, the agent checks in before the borrower is late:

- a friendly reminder with a payment link for borrowers who are probably just forgetful;
- an early check-in for borrowers who may be struggling, asking whether the date still works;
- an offer to align the schedule with the borrower's income, such as after harvest, on market days or after payday, within the playbook's limits.

Every offer goes through the same policy layer and realism check as any other plan.

Fair by rule. Regional and weather signals can only trigger help: an earlier check-in or a gentler schedule. They are never used for harsher treatment, more frequent contact or refusing anyone.

Recover: payment, receipts, negotiation and the realism check

When a borrower is late, the agent makes paying easy, knows when money arrives, negotiates within limits, and refuses plans that only delay the problem.

Payment links. Every reminder and agreed plan carries a link for the exact amount. Payments go through a small provider interface (create link, receive webhook, verify transaction). Chapa, an Ethiopian payment gateway, is the provider built for the demo, in test mode. The backend confirms every webhook with the gateway's verify call before updating the loan, and a unique reference per payment means a webhook delivered twice is recorded once. Other gateways, such as M-Pesa or Stripe, plug into the same interface.

Receipt checks. Many borrowers pay by mobile money or bank transfer and send a screenshot. Gemini reads the image and extracts the transaction number, amount, date, sender and receiving account. The agent then checks that the receiving account is the lender's, the amount and date fit the loan, the transaction number has never been used before, and the image is not a near-copy of an earlier receipt. A receipt that passes is matched as "verified by receipt"; one that fails goes to the officer with the reason.

Negotiation within the playbook. The lender sets the limits. Example values:

Rule	Example
Minimum first payment for a plan	30% of the overdue amount
Maximum extension without approval	14 days
Maximum instalments	3
Fee waivers	Manager only; never offered by the agent

The agent asks for full payment first, then asks what the borrower can pay and when, builds a plan from the answer, and makes one counter-offer at the limit if needed. Anything outside the playbook goes to a manager with a suggested plan.

Realism check. Before any plan is offered, code compares it with the borrower's history: past plans and whether they were kept, payment sizes, how late payments have been, and how many times this loan has been rescheduled. A plan that is unlikely to hold, or that would simply roll the problem forward again, is not offered. The case goes to the officer instead, flagged as a restructuring risk. This keeps the agent from hiding bad loans by rescheduling them.

Steer: forecast, policy sandbox and weekly report

The manager gets an advisor, not just a dashboard.

Cash forecast. Expected collections for the next 30, 60 and 90 days, built from each loan's schedule and its likelihood to pay, with a range rather than a single number. It answers the question a lender that depends on borrowed funding asks every month: what money is coming in, and when.

Policy sandbox. Before changing how the lender treats a group of borrowers, the manager can test it. The simulator runs the proposed policy on a copy of the loan book and compares it with current policy on recovery, cash timing, loans saved and loans that still go bad. Typical questions:

- What if farmers in a dry region get a 30-day extension?

- What if the minimum first payment for plans drops from 30% to 20%?
- What if reminders start 7 days before the due date instead of 3?

If the result is good, the agent drafts the playbook change and the manager approves it in one click.

Requests in plain language. The manager can hand the agent a problem rather than a query. Given "Farmers in Sidama are struggling; what should we do?", the agent finds the affected loans, proposes two or three policies, runs each in the sandbox, and recommends one with its numbers and the reasons. Nothing changes until the manager approves.

Weekly report. Every Monday the agent reports like an employee: what it collected, which loans it saved, which promises were kept, where risk is rising, what it handed over, and which decisions it needs from the manager.

Proof: simulator, stress test and evaluation suite

A lender should not have to take the agent on trust. The project measures it, and measures how much the measurement can be trusted.

Borrower simulator

A simulated loan book of a few hundred borrowers. Each has hidden traits the agent cannot see: income pattern, ability and willingness to pay, responsiveness, sensitivity to rainfall for farming loans, and a persona such as forgetful, struggling, evasive or reliable. Code decides what each borrower does from those traits, with a fixed random seed so runs are reproducible. Gemini plays borrowers in conversation, writing replies in character, including partial offers, excuses and broken promises.

The baseline is calibrated so that, with plain reminders, the share of loans going bad is in the range of published sector figures, such as the National Bank of Ethiopia's reported non-performing loan ratio for microfinance institutions.

The same loan book runs under two strategies:

- **Baseline:** fixed reminders before and on the due date, then every few days.
- **Agent:** Prevent, Recover and Steer as designed.

Results compared: collection rate, loans saved, loans gone bad, promise-kept rate, days to collect and messages per birr recovered.

Stress test

A simulator can only prove what it was built to assume. So the comparison is rerun under different assumptions:

Scenario	What changes
Weather matters	Rainfall strongly affects farming borrowers
Weather matters a little	Weak effect
Weather does not matter	No effect at all
Harder borrowers	More evasive and struggling personas
Noisy signals	Early warning signs are less reliable

The design goal is that the agent beats the baseline in every scenario, because negotiation, payment links and the realism check do not depend on the weather assumption. Wherever it does not, the report says so. That shows which parts of the agent carry the result, not only that it wins.

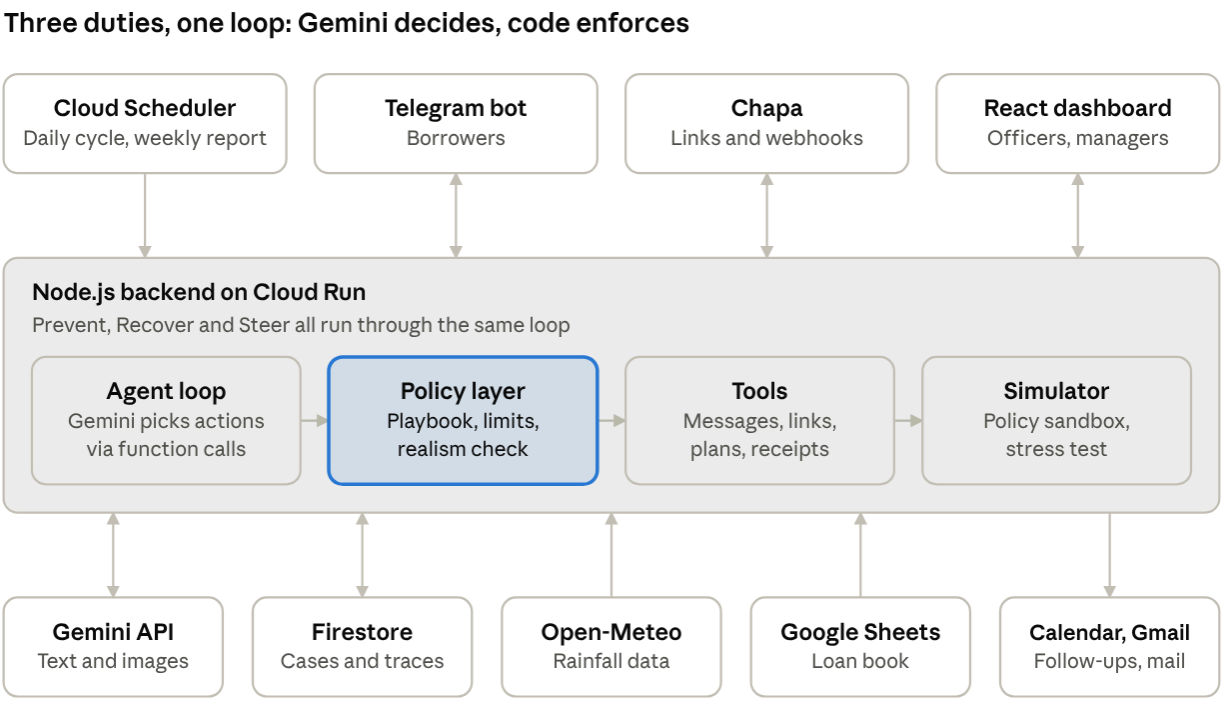
Evaluation suite

A labelled test set that runs in CI on every change:

- **Replies:** English, Amharic and mixed-language borrower messages with the correct intent, amounts and dates. Measures intent accuracy, extraction accuracy and false "paid" readings, the most costly error.
- **Receipts:** valid, reused, edited and mismatched receipt images. Measures detection of bad receipts and false alarms on good ones.
- **Policy and safety:** adversarial cases, such as a borrower asking for a fee waiver or telling the agent to ignore its rules, plus tone checks on generated messages.

A prompt or model change that lowers these scores fails the build, like a failing unit test.

Architecture



architecture · triggers, backend, services

Events come from four sources: the schedule, borrowers on Telegram, payment webhooks from Chapa, and staff on the dashboard. Every duty runs through the same loop: Gemini chooses an action, the policy layer checks it in code, and only then does a tool run it. The simulator is a tool too, used by the policy sandbox and the stress test on a copy of the data. Firestore holds the cases and the decision trace, Open-Meteo and Google Sheets feed data in, and the browser talks only to the backend, so no key reaches it.

Tech stack and APIs

One language end to end, free tiers only.

Layer	Choice	Why
Frontend	React, Vite, TypeScript, Tailwind, Recharts; Firebase Hosting	Responsive UI, charts for the forecast and simulator, free hosting
Backend	Node.js, TypeScript, Express on Cloud Run	Scales to zero; same language as the frontend
Shared contracts	Zod schemas in a shared package	One set of types validates API calls, Gemini output and the UI

Layer	Choice	Why
Scheduler	Cloud Scheduler	Starts the daily cycle and the weekly report
Database	Firestore	Free tier, live updates to the dashboard
Auth	Firebase Authentication	Officer and manager roles per lender
AI	Gemini API (Flash) via Google AI Studio	Function calling, JSON output against a schema, image reading for receipts
Secrets	Secret Manager into Cloud Run	No keys in code or in the browser
Quality	Vitest, evaluation suite, GitHub Actions CI	Tests and evaluation scores on every change

Third-party APIs (all free):

- **Telegram Bot API:** the borrower channel: messages and receipt photos in, replies out, through a webhook.
- **Chapa API (test mode):** payment links, success webhooks and transaction verification.
- **Open-Meteo API:** rainfall history and recent levels per region for the early-warning signals. No key needed.
- **Google Sheets API:** imports the lender's loan book.
- **Google Calendar API:** follow-ups for promises and plans on the officer's calendar.
- **Gmail API:** the officer's daily briefing, the manager's weekly report, and email reminders for borrowers who prefer it.

User interface

A responsive web app for officers and managers. Borrowers need no app: they use Telegram.

Officer:

- **Today:** expected collections, what has been paid, loans at rising risk, and a call list ranked by expected recovery. Each case has a short brief and a suggested approach.
- **Approval queue:** messages and plans waiting for a person, each with the agent's reasoning and the result of the realism check. Approve, edit or reject in one click.

- **Case view:** loan, schedule, signals, conversation, receipts, plans and payments, plus the decision trace for every action.
- **Receipts:** receipts the agent could not verify, with the reason, beside the matching loan.

Manager:

- **Portfolio:** money at risk, loans moving between lateness levels, and the regions and sectors where risk is rising.
- **Cash forecast:** expected collections for 30, 60 and 90 days, with a range.
- **Policy sandbox:** set up a policy, run it, and compare it with current policy side by side; approve it into the playbook.
- **Ask the agent:** problems in plain language; the agent shows its steps, the options it tested and its recommendation.
- **Agent performance:** the weekly report, targets over time, and the simulator and stress-test results.
- **Setup and playbook:** spreadsheet import with the column mapping, negotiation limits, autonomy level, quiet hours and contact limits.

Borrower, in Telegram: reminders and check-ins with payment links, replies in their own words, receipt photos, requests for a different schedule, and a request to speak to the officer.

Data model, safety and fairness

Main collections in Firestore, all scoped by lender so one deployment serves many lenders:

Collection	Holds
lenders, users	Settings, playbook, autonomy level; officers and managers with roles
borrowers	Contact, region, income pattern, preferred channel, language and time, opt-out, case summary
loans	Amount, schedule, balance, status, lateness level, risk score and reasons
signals	Per-region rainfall readings and per-loan behaviour signals, with dates
payments, receipts	Source, amount, reference, verification status; receipt images, extracted fields, check results

Collection	Holds
messages	Direction, channel, text, detected intent
plans, promises	Terms, dates, status, realism check result, approval record, Calendar event ids
agent_runs	Decision trace: inputs, tool calls, policy results, outcome
simulations	Policy tested, scenario, seed and results

Safety:

- **Hard limits in code.** Quiet hours, a daily contact cap, opt-out on "stop", playbook limits and the realism check. The model cannot override them.
- **Respectful messages.** Generated text is checked before sending: no threats, shaming or false claims, and the debt is never mentioned to anyone but the borrower.
- **People decide the hard parts.** Disputes, hardship, legal matters and anything outside the playbook go to a person.
- **Untrusted input stays data.** Borrower messages and receipt images are never treated as instructions; the evaluation suite tests prompt-injection cases.
- **Correct money handling.** Payments are recorded once, verified with the gateway, and never marked paid on a borrower's word alone.
- **Simulation is separate.** The sandbox works on a copy of the loan book and cannot change live loans.

Fairness:

- Regional and weather signals can only trigger help, never harsher treatment or refusal.
- Risk reasons are shown for every score, so an officer can see and challenge them.
- The dashboard reports outcomes by region, so any group being treated worse becomes visible.

Privacy: minimal personal data, role-based access, secrets in Secret Manager, and invented borrowers and test accounts only in the demo.

Build plan, scope and demo

Scope. The full design above is the product vision. The build delivers a complete, working core rather than every feature in part:

In the build	Kept simple	Roadmap
Daily cycle, risk score with reasons, early outreach with weather signals	Spreadsheet setup with column mapping	Learning which message versions work best
Payment links, webhooks and verification; receipt checks	Telegram as the only borrower channel	Amharic voice notes
Negotiation within the playbook; realism check; approval queue	Email for the briefing and weekly report	Bank statement import
Cash forecast; policy sandbox; manager requests	Weekly report from a template plus Gemini summary	Funder and regulator reports
Simulator, stress test, evaluation suite in CI		More payment providers and WhatsApp

Build order, so a working end-to-end slice exists early:

1. **Foundation:** monorepo with shared schemas, Firestore, auth and roles, spreadsheet import, CI.
2. **Core loop:** daily cycle, signals, risk score, policy layer, Telegram, decision trace.
3. **Money:** payment provider interface with Chapa, webhooks and verification, receipt checks.
4. **Negotiation:** playbook, plans, realism check, approval queue, Calendar follow-ups.
5. **Simulator:** borrower model, baseline, stress-test scenarios.
6. **Steer:** cash forecast, policy sandbox, manager requests, weekly report.
7. **Proof and polish:** evaluation suite in CI, agent-versus-baseline results, deploy, README.

Open point. Google's setup guides for Cloud Run and Cloud Scheduler ask for billing to be enabled. If that requires a card I cannot use, I will follow the assignment's note and ask for guidance.

Demo video:

1. **Problem:** losses start before collections does, and why small lenders are exposed.
2. **Setup:** a messy spreadsheet imported and mapped in minutes.
3. **Morning run:** Cloud Scheduler starts it; risk scores with reasons appear.
4. **Prevent:** Abebe is flagged before his due date, accepts a schedule matched to his harvest, and pays half through the link.

5. **Recover:** Sara negotiates and pays; a reused receipt is caught.
6. **Hand over:** Dawit's case goes to the officer because another plan would not hold.
7. **Steer:** the manager asks what to do about a dry region; the agent tests options in the sandbox and recommends one.
8. **Proof:** agent versus baseline across the stress-test scenarios, and the evaluation scores.
9. **Code walkthrough:** agent loop and tools, policy layer, payment interface, simulator, frontend structure and tests.

The source code, this document and setup instructions will be in a public GitHub repository.

Roadmap

What comes after the build, in order of value to a lender:

1. **Learning what works.** Run message tone, wording, channel and timing as continuous experiments, so the agent shifts towards what gets paid, within the versions the policy layer allows.
2. **A reviewer agent.** A second model checks sensitive messages and plans against the playbook and a fair-treatment checklist before they go out, with its catch rate measured in the evaluation suite.
3. **Group loans and guarantors.** Contact a group leader or guarantor only where the borrower agreed to it at signing, and only within what that agreement allows.
4. **More signals.** Market prices for the main crops, and regional events, added to the early-warning model only where they are shown to predict missed payments.
5. **Funder and regulator reports.** Portfolio reports generated from the same data, with figures computed in code.
6. **More channels and markets.** WhatsApp, Amharic and Afaan Oromo voice notes, and payment providers such as M-Pesa and Stripe through the existing interface.
7. **Field visits.** Group borrowers who need a visit by area and suggest a route for the field officer.

Sources

- Finezza: operational challenges for microfinance lenders: intervening at 15 rather than 75 days past due; manual tracking; officer turnover.
- Capital Ethiopia: Microfinance must go digital or face irrelevance: weather risk to repayment; manual, branch-based models.

- National Bank of Ethiopia: Financial Stability Report, March 2026: number of microfinance institutions, non-performing loan ratio, loans-to-deposits ratio.
- Digital Frontiers: Ethiopia's uncollateralised digital credit revolution: the risk of evergreening through constant restructuring.
- UNCDF: Consumer protection in the digital lending era: harsh collection practices in East African digital lending.
- YourStory: Prodigal's AI collections platform: existing AI collections products for large lenders.
- Kompato: AI in debt collection, 2026 guide: early intervention from regional signals described as an emerging capability.
- Chapa: accept payments: payment links, callbacks and verification.